List of standard data types

This section lists all standard data types. Link to the repository that contains the sources of all DSDL definitions can be found on the contacts page.

Autogenerated Tue, 26 Aug 2025 01:12:48 +0000.

uavcan

CoarseOrientation

Full name: uavcan.CoarseOrientation

```
#
# Nested type.
# Coarse, low-resolution 3D orientation represented as fixed axes in 16 bit.
#
# Roll, pitch, yaw angles in radians should be multiplied by
# ANGLE_MULTIPLIER in order to convert them to the coarse representation.
#
# ANGLE_MULTIPLIER = NORM / PI
#
# Where NORM is 12, because it:
#  - Fits the maximum range of a signed 5 bit integer
#  - Allows to exactly represent the following angles:
#    0, 15, 30, 45, 60, 75, 90, 105, 120, 135, 150, 165, 180, and negatives
#

float32 ANGLE_MULTIPLIER = 4.7746482927568605

int5[3] fixed_axis_roll_pitch_yaw

bool orientation_defined    # False if the orientation is actually not defined
```

Timestamp

Full name: uavcan.Timestamp

```
#
# Global timestamp in microseconds, 7 bytes.
#
# Use this data type for timestamp fields in messages, like follows:
#   uavcan.Timestamp timestamp
#

uint56 UNKNOWN = 0
truncated uint56 usec     # Microseconds
```

uavcan.equipment.actuator

ArrayCommand

Full name: uavcan.equipment.actuator.ArrayCommand

Default data type ID: 1010

Specification

1. Introduction
2. Basic concepts
3. Data structure description language
4.1 CAN bus transport layer
4.2 UDP bus transport layer
4.3 MAVLink bus transport layer
4.4 CANFD bus transport layer
5. Application level conventions
6. Application level functions
7. List of standard data types
8. Hardware design recommendations
dsdl
  ardupilot
  uavcan

Contents

List of standard data types
  uavcan
    CoarseOrientation
    Timestamp
  uavcan.equipment.actuator
    ArrayCommand
    Status
    Command
  uavcan.equipment.ahrs
    Solution
    MagneticFieldStrength
    MagneticFieldStrength2
    RawIMU
  uavcan.equipment.air_data
    TrueAirspeed
    IndicatedAirspeed
    AngleOfAttack
    Sideslip
    RawAirData
    StaticPressure
    StaticTemperature
  uavcan.equipment.camera_gimbal
    AngularCommand
    GEOPOICommand
    Status
    Mode
  uavcan.equipment.device

```
# The system supports up to 256 actuators; up to 15 of them can be commanded with one message.
#

Command[<=15] commands
```

Status

Full name: `uavcan.equipment.actuator.Status`

Default data type ID: 1011

```
#
# Generic actuator feedback, if available.
# Unknown fields should be set to NAN.
#

uint8 actuator_id

#
# Whether the units are linear or angular depends on the actuator type (refer to the Command da
#
float16 position        # meter or radian
float16 force           # Newton or Newton metre
float16 speed           # meter per second or radian per second

void1
uint7 POWER_RATING_PCT_UNKNOWN = 127
uint7 power_rating_pct                # 0 - unloaded, 100 - full load
```

Command

Full name: `uavcan.equipment.actuator.Command`

```
#
# Nested type.
# Single actuator command.
#

uint8 actuator_id

#
# Whether the units are linear or angular depends on the actuator type.
#
uint8 COMMAND_TYPE_UNITLESS     = 0     # [-1, 1]
uint8 COMMAND_TYPE_POSITION     = 1     # meter or radian
uint8 COMMAND_TYPE_FORCE        = 2     # Newton or Newton metre
uint8 COMMAND_TYPE_SPEED        = 3     # meter per second or radian per second
uint8 command_type

#
# Value of the above type
#
float16 command_value
```

uavcan.equipment.ahrs

Solution

Full name: `uavcan.equipment.ahrs.Solution`

Default data type ID: 1000

RawCommand
RPMCommand
Status
uavcan.equipment.gnss
Fix
Auxiliary
RTCMStream
Fix2
ECEFPositionVelocity
uavcan.equipment.hardpoint
Command
Status
uavcan.equipment.ice
FuelTankStatus
uavcan.equipment.ice.reciprocatin
Status
CylinderStatus
uavcan.equipment.indication
BeepCommand
LightsCommand
RGB565
SingleLightCommand
uavcan.equipment.power
PrimaryPowerSupplyStatus
CircuitStatus
BatteryInfo
uavcan.equipment.range_sensor
Measurement
uavcan.equipment.safety
ArmingStatus
uavcan.navigation
GlobalNavigationSolution
uavcan.protocol
GetNodeInfo
GetDataTypeInfo
NodeStatus
GetTransportStats
GlobalTimeSync
Panic
RestartNode
AccessCommandShell
CANIfaceStats
DataTypeKind
HardwareVersion
SoftwareVersion
uavcan.protocol.debug
KeyValue
LogMessage
LogLevel
uavcan.protocol.dynamic_node_id
Allocation
uavcan.protocol.dynamic_node_id.s
AppendEntries
RequestVote
Discovery
Entry
uavcan.protocol.enumeration

```
#

uavcan.Timestamp timestamp

#
# Normalized quaternion
#
float16[4] orientation_xyzw
void4
float16[<=9] orientation_covariance

#
# rad/sec
#
float16[3] angular_velocity
void4
float16[<=9] angular_velocity_covariance

#
# m/s^2
#
float16[3] linear_acceleration
float16[<=9] linear_acceleration_covariance
```

MagneticFieldStrength

Full name: uavcan.equipment.ahrs.MagneticFieldStrength

Default data type ID: 1001

```
#
# Magnetic field readings, in Gauss, in body frame.
# SI units are avoided because of float16 range limitations.
# This message is deprecated. Use the newer 1002.MagneticFieldStrength2.uavcan message.
#

float16[3] magnetic_field_ga
float16[<=9] magnetic_field_covariance
```

MagneticFieldStrength2

Full name: uavcan.equipment.ahrs.MagneticFieldStrength2

Default data type ID: 1002

```
#
# Magnetic field readings, in Gauss, in body frame.
# SI units are avoided because of float16 range limitations.
#

uint8 sensor_id

float16[3] magnetic_field_ga
float16[<=9] magnetic_field_covariance
```

RawIMU

Full name: uavcan.equipment.ahrs.RawIMU

Default data type ID: 1003

```
#
# Raw IMU data with timestamps.
```


uavcan.protocol.file
- BeginFirmwareUpdate
- GetInfo
- GetDirectoryEntryInfo
- Delete
- Read
- Write
- EntryType
- Error
- Path

uavcan.protocol.param
- ExecuteOpcode
- GetSet
- Empty
- NumericValue
- Value

uavcan.tunnel
- Broadcast
- Call
- Protocol

dronecan.sensors.hygrometer
- Hygrometer

ardupilot.equipment.power
- BatteryInfoAux

ardupilot.equipment.trafficmonito
- TrafficReport

ardupilot.gnss
- Heading
- Status
- MovingBaselineData
- RelPosHeading

ardupilot.indication
- SafetyState
- Button
- NotifyState

com.hex.equipment.flow
- Measurement

License

This work is licensed under a Creative Commons Attribution 4.0 International License.
Much of the content of this site is based upon prior work by Pavel Kirienko and the UAVCAN Development Team.

```
#
# Data acquisition timestamp in the bus shared time base.
#
uavcan.Timestamp timestamp

#
# Integration interval, seconds.
# Set to a non-positive value if the integrated samples are not available
# (in this case, only the latest point samples will be valid).
#
float32 integration_interval

#
# Angular velocity samples in radian/second.
# The samples are represented in the body frame, the axes are ordered as follows:
#   1. angular velocity around X (roll rate)
#   2. angular velocity around Y (pitch rate)
#   3. angular velocity around Z (yaw rate)
#
float16[3] rate_gyro_latest                # Latest sample, radian/second
float32[3] rate_gyro_integral              # Integrated samples, radian/second

#
# Linear acceleration samples in meter/(second^2).
# The samples are represented in the body frame, the axes are ordered as follows:
#   1. linear acceleration along X (forward positive)
#   2. linear acceleration along Y (right positive)
#   3. linear acceleration along Z (down positive)
#
float16[3] accelerometer_latest            # Latest sample, meter/(second^2)
float32[3] accelerometer_integral          # Integrated samples, meter/(second^2)

#
# Covariance matrix. The diagonal entries are ordered as follows:
#   1. roll rate               (radian^2)/(second^2)
#   2. pitch rate              (radian^2)/(second^2)
#   3. yaw rate                (radian^2)/(second^2)
#   4. forward acceleration    (meter^2)/(second^4)
#   5. rightward acceleration  (meter^2)/(second^4)
#   6. downward acceleration   (meter^2)/(second^4)
#
float16[<=36] covariance
```

uavcan.equipment.air_data

TrueAirspeed

Full name: `uavcan.equipment.air_data.TrueAirspeed`

Default data type ID: 1020

```
#
# TAS.
#

float16 true_airspeed           # m/s
float16 true_airspeed_variance  # (m/s)^2
```

IndicatedAirspeed

Full name: `uavcan.equipment.air_data.IndicatedAirspeed`

Default data type ID: 1021

```
#

float16 indicated_airspeed              # m/s
float16 indicated_airspeed_variance     # (m/s)^2
```

`AngleOfAttack`

Full name: `uavcan.equipment.air_data.AngleOfAttack`

Default data type ID: 1025

```
#
# Angle of attack.
#

uint8 SENSOR_ID_LEFT = 254
uint8 SENSOR_ID_RIGHT = 255
uint8 sensor_id

float16 aoa             # Radians
float16 aoa_variance    # Radians^2
```

`Sideslip`

Full name: `uavcan.equipment.air_data.Sideslip`

Default data type ID: 1026

```
#
# Body sideslip in radians.
#

float16 sideslip_angle          # Radians
float16 sideslip_angle_variance # Radians^2
```

`RawAirData`

Full name: `uavcan.equipment.air_data.RawAirData`

Default data type ID: 1027

```
#
# Raw Air Data.
#

# Note: unused vars should be assigned NaN

#
# Heater State
#
uint8 FLAG_HEATER_AVAILABLE      = 1
uint8 FLAG_HEATER_WORKING        = 2
uint8 FLAG_HEATER_OVERCURRENT    = 4
uint8 FLAG_HEATER_OPENCIRCUIT    = 8
uint8 flags

#
# Pressure Data
#
float32 static_pressure                 # Pascal
float32 differential_pressure           # Pascal

#
```

```
float16 differential_pressure_sensor_temperature    # Kelvin

float16 static_air_temperature          # Kelvin
                                        # This field contains the raw temperature reading
                                        # from the externally mounted temperature sensor or,
                                        # in absence of one, the raw temperature of the pressur

float16 pitot_temperature               # Kelvin


float16[<=16] covariance                # order of diagonal elements :
                                        # static_pressure, differential_pressure,
                                        # static_air_temperature, pitot_temperature
                                        # Pascal^2 for pressure variance and covariance
                                        # Kevin^2 for Temperature variance and covariance
                                        # Pascal*Kelvin for pressure/temperature covariance
```

`StaticPressure`

Full name: `uavcan.equipment.air_data.StaticPressure`

Default data type ID: 1028

```
#
# Static pressure.
#

float32 static_pressure                 # Pascal
float16 static_pressure_variance        # Pascal^2
```

`StaticTemperature`

Full name: `uavcan.equipment.air_data.StaticTemperature`

Default data type ID: 1029

```
#
# Static temperature.
#

float16 static_temperature              # Kelvin
float16 static_temperature_variance     # Kelvin^2
```

`uavcan.equipment.camera_gimbal`

`AngularCommand`

Full name: `uavcan.equipment.camera_gimbal.AngularCommand`

Default data type ID: 1040

```
#
# Generic camera gimbal control.
#
# This message can only be used in the following modes:
#  - COMMAND_MODE_ANGULAR_VELOCITY
#  - COMMAND_MODE_ORIENTATION_FIXED_FRAME
#  - COMMAND_MODE_ORIENTATION_BODY_FRAME
#

uint8 gimbal_id
```

```
# See the list of acceptable modes above.
#
Mode mode

#
# In the angular velocity mode, this field contains a rate quaternion.
# In the orientation mode, this field contains orientation either in fixed frame or in body fra
#
float16[4] quaternion_xyzw
```

`GEOPOICommand`

Full name: `uavcan.equipment.camera_gimbal.GEOPOICommand`

Default data type ID: 1041

```
#
# Generic camera gimbal control.
#
# This message can only be used in the following modes:
#  - COMMAND_MODE_GEO_POI
#

uint8 gimbal_id

#
# Target operation mode - how to handle this message.
# See the list of acceptable modes above.
#
Mode mode

#
# Coordinates of the POI (point of interest).
#
int32 longitude_deg_1e7    # 1 LSB = 1e-7 deg
int32 latitude_deg_1e7
int22 height_cm            # 1 LSB = 10 mm

uint2 HEIGHT_REFERENCE_ELLIPSOID = 0
uint2 HEIGHT_REFERENCE_MEAN_SEA_LEVEL = 1
uint2 height_reference
```

`Status`

Full name: `uavcan.equipment.camera_gimbal.Status`

Default data type ID: 1044

```
#
# Generic gimbal status.
#

uint8 gimbal_id

Mode mode

#
# Camera axis orientation in body frame (not in fixed frame).
# Please refer to the UAVCAN coordinate frame conventions.
#
float16[4] camera_orientation_in_body_frame_xyzw
float16[<=9] camera_orientation_in_body_frame_covariance   # +inf for non-existent axes
```

Mode

Full name: `uavcan.equipment.camera_gimbal.Mode`

```
#
# Gimbal operating mode
#

uint8 COMMAND_MODE_ANGULAR_VELOCITY        = 0
uint8 COMMAND_MODE_ORIENTATION_FIXED_FRAME = 1
uint8 COMMAND_MODE_ORIENTATION_BODY_FRAME  = 2
uint8 COMMAND_MODE_GEO_POI                 = 3
uint8 command_mode
```

uavcan.equipment.device

Temperature

Full name: `uavcan.equipment.device.Temperature`

Default data type ID: 1110

```
#
# Generic device temperature
#

uint16 device_id

float16 temperature                 # in kelvin

uint8 ERROR_FLAG_OVERHEATING = 1
uint8 ERROR_FLAG_OVERCOOLING = 2
uint8 error_flags
```

uavcan.equipment.esc

RawCommand

Full name: `uavcan.equipment.esc.RawCommand`

Default data type ID: 1030

```
#
# Raw ESC command normalized into [-8192, 8191]; negative values indicate reverse rotation.
# The ESC should normalize the setpoint into its effective input range.
# Non-zero setpoint value below minimum should be interpreted as min valid setpoint for the giv
#

int14[<=20] cmd
```

RPMCommand

Full name: `uavcan.equipment.esc.RPMCommand`

Default data type ID: 1031

```
#
# Simple RPM setpoint.
# The ESC should automatically clamp the setpoint according to the minimum and maximum supporte
```

```
int18[<=20] rpm
```

Status

Full name: `uavcan.equipment.esc.Status`

Default data type ID: 1034

```
#
# Generic ESC status.
# Unknown fields should be set to NAN.
#

uint32 error_count          # Resets when the motor restarts

float16 voltage             # Volt
float16 current             # Ampere. Can be negative in case of a regenerative braking.
float16 temperature         # Kelvin

int18 rpm                   # Negative value indicates reverse rotation

uint7 power_rating_pct      # Instant demand factor in percent (percent of maximum power); rang

uint5 esc_index
```

uavcan.equipment.gnss

Fix

Full name: `uavcan.equipment.gnss.Fix`

Default data type ID: 1060

```
#
# GNSS navigation solution with uncertainty.
# This message is deprecated. Use the newer 1063.Fix2.uavcan message.
#

uavcan.Timestamp timestamp         # Global network-synchronized time, if available, otherwise

#
# Time solution.
# Time standard (GPS, UTC, TAI, etc) is defined in the field below.
#
uavcan.Timestamp gnss_timestamp

#
# Time standard used in the GNSS timestamp field.
#
uint3 GNSS_TIME_STANDARD_NONE = 0  # Time is unknown
uint3 GNSS_TIME_STANDARD_TAI  = 1
uint3 GNSS_TIME_STANDARD_UTC  = 2
uint3 GNSS_TIME_STANDARD_GPS  = 3
uint3 gnss_time_standard

void5   # Reserved space

#
# If known, the number of leap seconds allows to perform conversions between some time standard
#
uint8 NUM_LEAP_SECONDS_UNKNOWN = 0
uint8 num_leap_seconds
```

```
int37 longitude_deg_1e8           # Longitude degrees multiplied by 1e8 (approx. 1 mm per LSB)
int37 latitude_deg_1e8            # Latitude degrees multiplied by 1e8 (approx. 1 mm per LSB o
int27 height_ellipsoid_mm         # Height above ellipsoid in millimeters
int27 height_msl_mm               # Height above mean sea level in millimeters

float16[3] ned_velocity           # NED frame (north-east-down) in meters per second

#
# Fix status
#
uint6 sats_used

uint2 STATUS_NO_FIX    = 0
uint2 STATUS_TIME_ONLY = 1
uint2 STATUS_2D_FIX    = 2
uint2 STATUS_3D_FIX    = 3
uint2 status

#
# Precision
#
float16 pdop

void4
float16[<=9] position_covariance   # m^2
float16[<=9] velocity_covariance   # (m/s)^2
```

`Auxiliary`

Full name: `uavcan.equipment.gnss.Auxiliary`

Default data type ID: 1061

```
#
# GNSS low priority auxiliary info.
# Unknown DOP parameters should be set to NAN.
#

float16 gdop
float16 pdop
float16 hdop
float16 vdop
float16 tdop
float16 ndop
float16 edop

uint7 sats_visible                 # All visible sats of all available GNSS (e.g. GPS, GLONA
uint6 sats_used                    # All used sats of all available GNSS
```

`RTCMStream`

Full name: `uavcan.equipment.gnss.RTCMStream`

Default data type ID: 1062

```
#
# GNSS RTCM SC-104 protocol raw stream container.
# RTCM messages that are longer than max data size can be split over multiple consecutive messa
#

uint8 PROTOCOL_ID_UNKNOWN = 0
uint8 PROTOCOL_ID_RTCM2   = 2
uint8 PROTOCOL_ID_RTCM3   = 3
uint8 protocol_id
```

Fix2

Full name: `uavcan.equipment.gnss.Fix2`

Default data type ID: 1063

```
#
# GNSS ECEF and LLA navigation solution with uncertainty.
#

#
# Global network-synchronized time, if available, otherwise zero.
#
uavcan.Timestamp timestamp

#
# Time solution.
# The method and number of leap seconds which were in use for deriving the timestamp are
# defined in the fields below.
#
uavcan.Timestamp gnss_timestamp

#
# Method used for deriving the GNSS timestamp field.
# This data type relies on the following definitions:
#
#   Leap seconds  - Accumulated one-second adjustments applied to UTC since 1972.
#                   For reference, on May 2017, the number of leap seconds was equal 27.
#                   The number of leap seconds is taken from the field num_leap_seconds.
#                   Refer to https://en.wikipedia.org/wiki/Leap_second for a general overview.
#
#   TAI timestamp - The number of microseconds between the current TAI time and
#                   the TAI time at UTC 1970-01-01T00:00:00.
#
#   UTC timestamp - The number of microseconds between the current UTC time and
#                   UTC 1970-01-01T00:00:00.
#                   UTC can be expressed via TAI as follows (in seconds):
#                       UTC = TAI - num_leap_seconds - 10
#                   And via GPS (in seconds):
#                       UTC = GPS - num_leap_seconds + 9
#
#   GPS timestamp - The number of microseconds between the current GPS time and
#                   the GPS time at UTC 1970-01-01T00:00:00.
#                   GPS time can be expressed via TAI as follows (in seconds):
#                       GPS = TAI - 19
#
uint3 GNSS_TIME_STANDARD_NONE = 0  # Time is unknown
uint3 GNSS_TIME_STANDARD_TAI  = 1
uint3 GNSS_TIME_STANDARD_UTC  = 2
uint3 GNSS_TIME_STANDARD_GPS  = 3
uint3 gnss_time_standard

void13   # Reserved space

#
# Accumulated one-second adjustments applied to UTC since 1972.
# The number must agree with the currently correct number of UTC leap seconds. If this cannot
# be garanteed, the field must be set to NUM_LEAP_SECONDS_UNKNOWN.
#
uint8 NUM_LEAP_SECONDS_UNKNOWN = 0
uint8 num_leap_seconds

#
# Position and velocity solution
#
int37 longitude_deg_1e8            # Longitude degrees multiplied by 1e8 (approx. 1 mm per LSB)
int37 latitude_deg_1e8             # Latitude degrees multiplied by 1e8 (approx. 1 mm per LSB o
int27 height_ellipsoid_mm          # Height above ellipsoid in millimeters
int27 height_msl_mm                # Height above mean sea level in millimeters
```

```
# Fix status
#
uint6 sats_used

uint2 STATUS_NO_FIX    = 0
uint2 STATUS_TIME_ONLY = 1
uint2 STATUS_2D_FIX    = 2
uint2 STATUS_3D_FIX    = 3
uint2 status

#
# GNSS Mode
#
uint4 MODE_SINGLE      = 0
uint4 MODE_DGPS        = 1
uint4 MODE_RTK         = 2
uint4 MODE_PPP         = 3
uint4 mode

#
# GNSS Sub mode
#
uint6 SUB_MODE_DGPS_OTHER    = 0
uint6 SUB_MODE_DGPS_SBAS     = 1

uint6 SUB_MODE_RTK_FLOAT     = 0
uint6 SUB_MODE_RTK_FIXED     = 1

uint6 sub_mode

#
# Precision
#
float16[<=36] covariance    # Position and velocity covariance. Units are
                            # m^2 for position, (m/s)^2 for velocity and
                            # m^2/s for position/velocity.

float16 pdop

#
# Position and velocity solution in ECEF, if available
#
ECEFPositionVelocity[<=1] ecef_position_velocity
```

ECEFPositionVelocity

Full name: `uavcan.equipment.gnss.ECEFPositionVelocity`

```
#
# Nested type.
# GNSS ECEF high resolution position and velocity.
#
# ECEF is an acronym for Earth-Centered-Earth-Fixed, which is a cartesian
# coordinate system which rotates with the earth. The origin (0,0,0) is
# located at the center of the earth. The x-axis is a vector pointing from
# the origin with positive direction towards 0 degrees latitude and
# longitude (equator, at the prime meridian). The z-axis is a vector
# pointing from the origin towards the north-pole. The y-axis completes a
# right-handed coordinate system.
#

float32[3] velocity_xyz            # XYZ velocity in m/s

int36[3] position_xyz_mm           # XYZ-axis coordinates in mm

void6                              # Aligns the following array at byte boundary

float16[<=36] covariance           # Position and velocity covariance in the ECEF frame. Units
```

uavcan.equipment.hardpoint

Command

Full name: uavcan.equipment.hardpoint.Command

Default data type ID: 1070

```
#
# Generic cargo holder/hardpoint command.
#

uint8 hardpoint_id

#
# Either a binary command (0 - release, 1+ - hold) or bitmask
#
uint16 command
```

Status

Full name: uavcan.equipment.hardpoint.Status

Default data type ID: 1071

```
#
# Generic cargo holder/hardpoint status.
#

uint8 hardpoint_id

float16 payload_weight           # Newton
float16 payload_weight_variance

#
# Meaning is the same as for the command field in the Command message
#
uint16 status
```

uavcan.equipment.ice

FuelTankStatus

Full name: uavcan.equipment.ice.FuelTankStatus

Default data type ID: 1129

```
#
# Generic fuel tank status message.
# All fields are required unless stated otherwise. Unpopulated optional fields should be set to
#

#
# Reserved for future use.
#
void9

#
# The estimated amount of fuel.
# The reported values can be either measured directly using appropriate sensors,
# or they can be estimated by fusing the data provided by various sensors.
```

```
uint7 available_fuel_volume_percent     # Unit: percent, from 0% to 100%
float32 available_fuel_volume_cm3       # Unit: centimeter^3

#
# Estimate of the current fuel consumption rate.
# The flow can be negative if the fuel is being transferred between the tanks or during refueli
# This field is required.
# Unit: (centimeter^3)/minute
#
float32 fuel_consumption_rate_cm3pm

#
# Fuel temperature.
# This field is optional, set to NaN if not provided.
# Unit: kelvin
#
float16 fuel_temperature

#
# The ID of the current fuel tank.
#
uint8 fuel_tank_id
```

uavcan.equipment.ice.reciprocating

Status

Full name: `uavcan.equipment.ice.reciprocating.Status`

Default data type ID: 1120

```
#
# Generic status message of a piston engine control system.
#
# All integer fields are required unless stated otherwise.
# All floating point fields are optional unless stated otherwise; unknown/unapplicable fields s
#

#
# Abstract engine state. The flags defined below can provide further elaboration.
# This is a required field.
#
uint2 state
#
# The engine is not running. This is the default state.
# Next states: STARTING, FAULT
#
uint2 STATE_STOPPED = 0
#
# The engine is starting. This is a transient state.
# Next states: STOPPED, RUNNING, FAULT
#
uint2 STATE_STARTING = 1
#
# The engine is running normally.
# Some error flags may be set to indicate non-fatal issues, e.g. overheating.
# Next states: STOPPED, FAULT
#
uint2 STATE_RUNNING = 2
#
# The engine can no longer function.
# The error flags may contain additional information about the nature of the fault.
# Next states: STOPPED.
#
uint2 STATE_FAULT = 3

#
# General status flags.
```

```
# All unused bits must be cleared.
#
uint30 flags
#
# General error. This flag is required, and it can be used to indicate an error condition
# that does not fit any of the other flags.
# Note that the vendor may also report additional status information via the vendor specific st
# field of the NodeStatus message.
#
uint30 FLAG_GENERAL_ERROR                       = 1
#
# Error of the crankshaft sensor. This flag is optional.
#
uint30 FLAG_CRANKSHAFT_SENSOR_ERROR_SUPPORTED   = 2
uint30 FLAG_CRANKSHAFT_SENSOR_ERROR             = 4
#
# Temperature levels. These flags are optional; either none of them or all of them are supporte
#
uint30 FLAG_TEMPERATURE_SUPPORTED               = 8
uint30 FLAG_TEMPERATURE_BELOW_NOMINAL           = 16      # Under-temperature warning
uint30 FLAG_TEMPERATURE_ABOVE_NOMINAL           = 32      # Over-temperature warning
uint30 FLAG_TEMPERATURE_OVERHEATING             = 64      # Critical overheating
uint30 FLAG_TEMPERATURE_EGT_ABOVE_NOMINAL       = 128     # Exhaust gas over-temperature warnin
#
# Fuel pressure. These flags are optional; either none of them or all of them are supported.
#
uint30 FLAG_FUEL_PRESSURE_SUPPORTED             = 256
uint30 FLAG_FUEL_PRESSURE_BELOW_NOMINAL         = 512     # Under-pressure warning
uint30 FLAG_FUEL_PRESSURE_ABOVE_NOMINAL         = 1024    # Over-pressure warning
#
# Detonation warning. This flag is optional.
# This warning is cleared immediately after broadcasting is done if detonation is no longer hap
#
uint30 FLAG_DETONATION_SUPPORTED                = 2048
uint30 FLAG_DETONATION_OBSERVED                 = 4096    # Detonation condition observed warni
#
# Misfire warning. This flag is optional.
# This warning is cleared immediately after broadcasting is done if misfire is no longer happen
#
uint30 FLAG_MISFIRE_SUPPORTED                   = 8192
uint30 FLAG_MISFIRE_OBSERVED                    = 16384   # Misfire condition observed warning
#
# Oil pressure. These flags are optional; either none of them or all of them are supported.
#
uint30 FLAG_OIL_PRESSURE_SUPPORTED              = 32768
uint30 FLAG_OIL_PRESSURE_BELOW_NOMINAL          = 65536   # Under-pressure warning
uint30 FLAG_OIL_PRESSURE_ABOVE_NOMINAL          = 131072  # Over-pressure warning
#
# Debris warning. This flag is optional.
#
uint30 FLAG_DEBRIS_SUPPORTED                    = 262144
uint30 FLAG_DEBRIS_DETECTED                     = 524288  # Detection of debris warning

#
# Reserved space
#
void16

#
# Engine load estimate.
# Unit: percent.
# Range: [0, 127].
#
uint7 engine_load_percent

#
# Engine speed.
# Unit: revolutions per minute.
#
uint17 engine_speed_rpm

#
```

```
float16 spark_dwell_time_ms

#
# Atmospheric (barometric) pressure.
# Unit: kilopascal.
#
float16 atmospheric_pressure_kpa

#
# Engine intake manifold pressure.
# Unit: kilopascal.
#
float16 intake_manifold_pressure_kpa

#
# Engine intake manifold temperature.
# Unit: kelvin.
#
float16 intake_manifold_temperature

#
# Engine coolant temperature.
# Unit: kelvin.
#
float16 coolant_temperature

#
# Oil pressure.
# Unit: kilopascal.
#
float16 oil_pressure

#
# Oil temperature.
# Unit: kelvin.
#
float16 oil_temperature

#
# Fuel pressure.
# Unit: kilopascal.
#
float16 fuel_pressure

#
# Instant fuel consumption estimate.
# The estimated value should be low-pass filtered in order to prevent aliasing effects.
# Unit: (centimeter^3)/minute.
#
float32 fuel_consumption_rate_cm3pm

#
# Estimate of the consumed fuel since the start of the engine.
# This variable MUST be reset when the engine is stopped.
# Unit: centimeter^3.
#
float32 estimated_consumed_fuel_volume_cm3

#
# Throttle position.
# Unit: percent.
#
uint7 throttle_position_percent

#
# The index of the publishing ECU.
#
uint6 ecu_index

#
# Spark plug activity report.
# Can be used during pre-flight tests of the spark subsystem.
```

```
uint3 SPARK_PLUG_SINGLE         = 0
uint3 SPARK_PLUG_FIRST_ACTIVE   = 1
uint3 SPARK_PLUG_SECOND_ACTIVE  = 2
uint3 SPARK_PLUG_BOTH_ACTIVE    = 3

#
# Per-cylinder status information.
#
CylinderStatus[<=16] cylinder_status
```

CylinderStatus

Full name: uavcan.equipment.ice.reciprocating.CylinderStatus

```
#
# Cylinder state information.
# This is a nested data type.
#
# All unknown parameters should be set to NaN.
#

#
# Cylinder ignition timing.
# Units: angular degrees of the crankshaft.
#
float16 ignition_timing_deg

#
# Fuel injection time.
# Units: millisecond.
#
float16 injection_time_ms

#
# Cylinder head temperature (CHT).
# Units: kelvin.
#
float16 cylinder_head_temperature

#
# Exhaust gas temperature (EGT).
# Set to NaN if this cylinder is not equipped with an EGT sensor.
# Set this field to the same value for all cylinders if there is a single shared EGT sensor.
# Units: kelvin.
#
float16 exhaust_gas_temperature

#
# Estimated lambda coefficient.
# This parameter is mostly useful for monitoring and tuning purposes.
# Unit: dimensionless ratio
#
float16 lambda_coefficient
```

uavcan.equipment.indication

BeepCommand

Full name: uavcan.equipment.indication.BeepCommand

Default data type ID: 1080

```
#
# Nodes that are capable of producing sounds should obey.
```

```
float16 duration   # Sec
```

LightsCommand

Full name: `uavcan.equipment.indication.LightsCommand`

Default data type ID: 1081

```
#
# Lights control command.
#

SingleLightCommand[<=20] commands
```

RGB565

Full name: `uavcan.equipment.indication.RGB565`

```
#
# Nested type.
# RGB color in the standard 5-6-5 16-bit palette.
# Monocolor lights should interpret this as brightness setpoint: from zero (0, 0, 0) to full br
#

uint5 red
uint6 green
uint5 blue
```

SingleLightCommand

Full name: `uavcan.equipment.indication.SingleLightCommand`

```
#
# Nested type.
# Controls single light source, color or monochrome.
#

#
# Common aircraft lights IDs
#

# inform the crew working on the apron around noisy airplanes, wearing hearing protection,
# that the engines are turned on. Also called beacon light
uint8 LIGHT_ID_ANTI_COLLISION = 246

# a red light is mounted on the left, or port, side of the craft and a green on the right,
# or starboard, side both 110 degree, and tail white light of 140 degree. Also called navigatio
uint8 LIGHT_ID_RIGHT_OF_WAY   = 247

# high-intensity burst of white light, to help other pilots recognize the
# aircraft's position in low-visibility conditions
uint8 LIGHT_ID_STROBE         = 248

# positioned on the outer side just in front of the engine cowlings on the fuselage
uint8 LIGHT_ID_WING           = 249

# lights that highlite on the logo painted on the tail or other visible surface.
# Also called vertical tail flood lights
uint8 LIGHT_ID_LOGO           = 250

# help the pilots see the area in front of them and also shows other traffic that they're on th
uint8 LIGHT_ID_TAXI           = 251
```

```
# very bright, lights up the area in front but a lot more than the taxi light
uint8 LIGHT_ID_TAKE_OFF       = 253

# very bright lights on the wings to help the pilots during landing by
# lighting up the area where they're going to touch down
uint8 LIGHT_ID_LANDING        = 254

# usually yellow electroluminescent lightstrips designed to use
# during formation flying at night or under low visibility conditions
uint8 LIGHT_ID_FORMATION      = 255

uint8 light_id

RGB565 color      # Monocolor lights should interpret this as brightness
```

uavcan.equipment.power

PrimaryPowerSupplyStatus

Full name: `uavcan.equipment.power.PrimaryPowerSupplyStatus`

Default data type ID: 1090

```
#
# Primary power supply status.
# Typical publishing rate should be around 1~2 Hz.
#

#
# How many hours left to full discharge at average load over the last 10 seconds.
#
float16 hours_to_empty_at_10sec_avg_power               # [Hours]
float16 hours_to_empty_at_10sec_avg_power_variance      # [Hours^2]

#
# True if the publishing node senses that an external power source can be used, e.g. to charge
#
bool external_power_available

#
# Remaining energy estimate in percent.
#
uint7 remaining_energy_pct              # [Percent]     Required
uint7 remaining_energy_pct_stdev        # [Percent]     Error standard deviation. Use best gues
```

CircuitStatus

Full name: `uavcan.equipment.power.CircuitStatus`

Default data type ID: 1091

```
#
# Generic electrical circuit info.
#

uint16 circuit_id

float16 voltage
float16 current

uint8 ERROR_FLAG_OVERVOLTAGE  = 1
uint8 ERROR_FLAG_UNDERVOLTAGE = 2
uint8 ERROR_FLAG_OVERCURRENT  = 4
```

`BatteryInfo`

Full name: `uavcan.equipment.power.BatteryInfo`

Default data type ID: 1092

```
#
# Single battery info.
#
# Typical publishing rate should be around 0.2~1 Hz.
#
# Please refer to the Smart Battery data specification for some elaboration.
#

#
# Primary parameters.
# Some fields can be set to NAN if their values are unknown.
# Full charge capacity is expected to slowly reduce as the battery is aging. Normally its estim
# every charging cycle.
#
float16 temperature             # [Kelvin]
float16 voltage                 # [Volt]
float16 current                 # [Ampere]
float16 average_power_10sec     # [Watt]        Average power consumption over the last 10 seco
float16 remaining_capacity_wh   # [Watt hours]  Will be increasing during charging
float16 full_charge_capacity_wh # [Watt hours]  Predicted battery capacity when it is fully cha
float16 hours_to_full_charge    # [Hours]       Charging is expected to complete in this time;

#
# Status flags.
# Notes:
#  - CHARGING must be always set as long as the battery is connected to a charger, even if the
#  - CHARGED must be cleared immediately when the charger is disconnected.
#
uint11 STATUS_FLAG_IN_USE       = 1     # The battery is currently used as a power supply
uint11 STATUS_FLAG_CHARGING     = 2     # Charger is active
uint11 STATUS_FLAG_CHARGED      = 4     # Charging complete, but the charger is still active
uint11 STATUS_FLAG_TEMP_HOT     = 8     # Battery temperature is above normal
uint11 STATUS_FLAG_TEMP_COLD    = 16    # Battery temperature is below normal
uint11 STATUS_FLAG_OVERLOAD     = 32    # Safe operating area violation
uint11 STATUS_FLAG_BAD_BATTERY  = 64    # This battery should not be used anymore (e.g. low SOH
uint11 STATUS_FLAG_NEED_SERVICE = 128   # This battery requires maintenance (e.g. balancing, fu
uint11 STATUS_FLAG_BMS_ERROR    = 256   # Battery management system/controller error, smart bat
uint11 STATUS_FLAG_RESERVED_A   = 512   # Keep zero
uint11 STATUS_FLAG_RESERVED_B   = 1024  # Keep zero
uint11 status_flags

#
# State of Health (SOH) estimate, in percent.
# http://en.wikipedia.org/wiki/State_of_health
#
uint7 STATE_OF_HEALTH_UNKNOWN = 127     # Use this constant if SOH cannot be estimated
uint7 state_of_health_pct               # Health of the battery, in percent, optional

#
# Relative State of Charge (SOC) estimate, in percent.
# http://en.wikipedia.org/wiki/State_of_charge
#
uint7 state_of_charge_pct               # Percent of the full charge [0, 100]. This field is re
uint7 state_of_charge_pct_stdev         # SOC error standard deviation; use best guess if unkno

#
# Battery identification.
# Model instance ID must be unique within the same battery model name.
# Model name is a human-readable string that normally should include the vendor name, model nam
# type of this battery. This field should be assumed case-insensitive. Example: "Zubax Smart Ba
#
uint8 battery_id                        # Identifies the battery within this vehicle, e.g. 0 -
uint32 model_instance_id                # Set to zero if not applicable
```

uavcan.equipment.range_sensor

Measurement

Full name: `uavcan.equipment.range_sensor.Measurement`

Default data type ID: 1050

```
#
# Generic narrow-beam range sensor data.
#

uavcan.Timestamp timestamp

uint8 sensor_id

uavcan.CoarseOrientation beam_orientation_in_body_frame

float16 field_of_view                # Radians


uint5 SENSOR_TYPE_UNDEFINED = 0
uint5 SENSOR_TYPE_SONAR     = 1
uint5 SENSOR_TYPE_LIDAR     = 2
uint5 SENSOR_TYPE_RADAR     = 3
uint5 sensor_type

uint3 READING_TYPE_UNDEFINED   = 0   # Range is unknown
uint3 READING_TYPE_VALID_RANGE = 1   # Range field contains valid distance
uint3 READING_TYPE_TOO_CLOSE   = 2   # Range field contains min range for the sensor
uint3 READING_TYPE_TOO_FAR     = 3   # Range field contains max range for the sensor
uint3 reading_type

float16 range                        # Meters
```

uavcan.equipment.safety

ArmingStatus

Full name: `uavcan.equipment.safety.ArmingStatus`

Default data type ID: 1100

```
#
# This message represents the system arming status.
# Some nodes may refuse to operate unless the system is fully armed.
#

uint8 STATUS_DISARMED           = 0
uint8 STATUS_FULLY_ARMED        = 255

uint8 status
```

uavcan.navigation

GlobalNavigationSolution

Full name: `uavcan.navigation.GlobalNavigationSolution`

Default data type ID: 2000

```
#
# Fields marked as optional should be set to NaN if the corresponding value is unknown.
#

#
# Global network synchronized timestamp, if known.
# Set to zero if the timestamp is not known.
#
uavcan.Timestamp timestamp

#
# Geo location [angular degree].
#
float64 longitude                   # required
float64 latitude                    # required

#
# Height estimates [meter].
#
float32 height_ellipsoid            # Above ellipsoid (required)
float32 height_msl                  # Above the mean sea level (required)
float32 height_agl                  # Above ground level (provided by radar altimeter or LIDAR)
float32 height_baro                 # Barometric height (optional)

#
# Atmospheric pressure adjusted to sea level [hectopascal].
#
float16 qnh_hpa                     # optional

#
# Rotation quaternion between the NED frame and the body frame.
# Zero rotation corresponds to the following orientation:
#   X facing north
#   Y facing east
#   Z facing down
#
float32[4] orientation_xyzw

#
# Column order:
#   longitude                                 [meter^2]
#   latitude                                  [meter^2]
#   height (MSL or ellipsoid, whichever worse)  [meter^2]
#   roll angle                                [radian^2]
#   pitch angle                               [radian^2]
#   yaw angle                                 [radian^2]
#
float16[<=36] pose_covariance

#
# Linear velocity in the body frame, X-Y-Z [meter/second].
#
float32[3] linear_velocity_body

#
# Angular velocity in the body frame, roll-pitch-yaw [radian/second].
#
float32[3] angular_velocity_body

#
# Low resolution estimate of the linear acceleration in the body frame [(meter/second)^2].
# This estimate should be properly downsampled in order to avoid aliasing effects.
#
float16[3] linear_acceleration_body

#
# Column order:
#   X velocity      [(meter/second)^2]
#   Y velocity      [(meter/second)^2]
#   Z velocity      [(meter/second)^2]
#   roll velocity   [(radian/second)^2]
#   pitch velocity  [(radian/second)^2]
```

uavcan.protocol

GetNodeInfo

Full name: `uavcan.protocol.GetNodeInfo`

Default data type ID: 1

```
#
# Full node info request.
# Note that all fields of the response section are byte-aligned.
#

---

#
# Current node status
#
NodeStatus status

#
# Version information shall not be changed while the node is running.
#
SoftwareVersion software_version
HardwareVersion hardware_version

#
# Human readable non-empty ASCII node name.
# Node name shall not be changed while the node is running.
# Empty string is not a valid node name.
# Allowed characters are: a-z (lowercase ASCII letters) 0-9 (decimal digits) . (dot) - (dash) _
# Node name is a reversed internet domain name (like Java packages), e.g. "com.manufacturer.pro
#
uint8[<=80] name
```

GetDataTypeInfo

Full name: `uavcan.protocol.GetDataTypeInfo`

Default data type ID: 2

```
#
# Get the implementation details of a given data type.
#
# Request is interpreted as follows:
#  - If the field 'name' is empty, the fields 'kind' and 'id' will be used to identify the data
#  - If the field 'name' is non-empty, it will be used to identify the data type; the
#    fields 'kind' and 'id' will be ignored.
#

uint16 id                   # Ignored if 'name' is non-empty
DataTypeKind kind           # Ignored if 'name' is non-empty

uint8[<=80] name            # Full data type name, e.g. "uavcan.protocol.GetDataTypeInfo"

---

uint64 signature            # Data type signature; valid only if the data type is known (see FL

uint16 id                   # Valid only if the data type is known (see FLAG_KNOWN)
DataTypeKind kind           # Ditto

uint8 FLAG_KNOWN      = 1   # This data type is defined
```

```
uint8 flags

uint8[<=80] name            # Full data type name
```

NodeStatus

Full name: `uavcan.protocol.NodeStatus`

Default data type ID: 341

```
#
# Abstract node status information.
#
# All UAVCAN nodes are required to publish this message periodically.
#

#
# Publication period may vary within these limits.
# It is NOT recommended to change it at run time.
#
uint16 MAX_BROADCASTING_PERIOD_MS = 1000
uint16 MIN_BROADCASTING_PERIOD_MS = 2

#
# If a node fails to publish this message in this amount of time, it should be considered offli
#
uint16 OFFLINE_TIMEOUT_MS = 3000

#
# Uptime counter should never overflow.
# Other nodes may detect that a remote node has restarted when this value goes backwards.
#
uint32 uptime_sec

#
# Abstract node health.
#
uint2 HEALTH_OK         = 0     # The node is functioning properly.
uint2 HEALTH_WARNING    = 1     # A critical parameter went out of range or the node encountere
uint2 HEALTH_ERROR      = 2     # The node encountered a major failure.
uint2 HEALTH_CRITICAL   = 3     # The node suffered a fatal malfunction.
uint2 health

#
# Current mode.
#
# Mode OFFLINE can be actually reported by the node to explicitly inform other network
# participants that the sending node is about to shutdown. In this case other nodes will not
# have to wait OFFLINE_TIMEOUT_MS before they detect that the node is no longer available.
#
# Reserved values can be used in future revisions of the specification.
#
uint3 MODE_OPERATIONAL      = 0         # Normal operating mode.
uint3 MODE_INITIALIZATION   = 1         # Initialization is in progress; this mode is entered i
uint3 MODE_MAINTENANCE      = 2         # E.g. calibration, the bootloader is running, etc.
uint3 MODE_SOFTWARE_UPDATE  = 3         # New software/firmware is being loaded.
uint3 MODE_OFFLINE          = 7         # The node is no longer available.
uint3 mode

#
# Not used currently, keep zero when publishing, ignore when receiving.
#
uint3 sub_mode

#
# Optional, vendor-specific node status code, e.g. a fault code or a status bitmask.
#
```

GetTransportStats

Full name: `uavcan.protocol.GetTransportStats`

Default data type ID: 4

```
#
# Get transport statistics.
#

---

#
# UAVCAN transport layer statistics.
#
uint48 transfers_tx             # Number of transmitted transfers.
uint48 transfers_rx             # Number of received transfers.
uint48 transfer_errors          # Number of errors detected in the UAVCAN transport layer.

#
# CAN bus statistics, for each interface independently.
#
CANIfaceStats[<=3] can_iface_stats
```

GlobalTimeSync

Full name: `uavcan.protocol.GlobalTimeSync`

Default data type ID: 4

```
#
# Global time synchronization.
# Any node that publishes timestamped data must use this time reference.
#
# Please refer to the specification to learn about the synchronization algorithm.
#

#
# Broadcasting period must be within this range.
#
uint16 MAX_BROADCASTING_PERIOD_MS = 1100            # Milliseconds
uint16 MIN_BROADCASTING_PERIOD_MS = 40              # Milliseconds

#
# Synchronization slaves may switch to a new source if the current master was silent for this a
#
uint16 RECOMMENDED_BROADCASTER_TIMEOUT_MS = 2200    # Milliseconds

#
# Time in microseconds when the PREVIOUS GlobalTimeSync message was transmitted.
# If this message is the first one, this field must be zero.
#
truncated uint56 previous_transmission_timestamp_usec # Microseconds
```

Panic

Full name: `uavcan.protocol.Panic`

Default data type ID: 5

```
#
# This message may be published periodically to inform network participants that the system has
# an unrecoverable fault and is not capable of further operation.
```

```
# undertaking any emergency actions.
#

uint8 MIN_MESSAGES = 3

uint16 MAX_INTERVAL_MS = 500

#
# Short description that would fit a single CAN frame.
#
uint8[<=7] reason_text
```

`RestartNode`

Full name: `uavcan.protocol.RestartNode`

Default data type ID: 5

```
#
# Restart the node.
#
# Some nodes may require restart before the new configuration will be applied.
#
# The request should be rejected if magic_number does not equal MAGIC_NUMBER.
#

uint40 MAGIC_NUMBER = 0xACCE551B1E
uint40 magic_number

---

bool ok
```

`AccessCommandShell`

Full name: `uavcan.protocol.AccessCommandShell`

Default data type ID: 6

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This service allows to execute arbitrary commands on the remote node's internal system shell.
#
# Essentially, this service mimics a typical terminal emulator, with one text input (stdin) and
# outputs (stdout and stderr). When there's no process running, the input is directed into the
# handler itself, which interprets it. If there's a process running, the input will be directed
# stdin of the running process. It is possible to forcefully return the terminal into a known s
# means of setting the reset flag (see below), in which case the terminal will kill all of the
# processes, if any, and return into the initial idle state.
#
# The server is assumed to allocate one independent terminal instance per client, so that diffe
# can execute commands without interfering with each other.
#

#
# Input and output should use this newline character.
#
uint8 NEWLINE = '\n'

#
# The server is required to keep the result of the last executed command for at least this time
# When this time expires, the server may remove the results in order to reclaim the memory, but
# is not guaranteed. Hence, the clients must retrieve the results in this amount of time.
#
uint8 MIN_OUTPUT_LIFETIME_SEC = 10
```

```
#
uint8 FLAG_RESET_SHELL          = 1     # Restarts the shell instance anew; may or may not impl
uint8 FLAG_CLEAR_OUTPUT_BUFFERS = 2     # Makes stdout and stderr buffers empty
uint8 FLAG_READ_STDOUT          = 64    # Output will contain stdout
uint8 FLAG_READ_STDERR          = 128   # Output will be extended with stderr
uint8 flags

#
# If the shell is idle, it will interpret this string.
# If there's a process running, this string will be piped into its stdin.
#
# If RESET_SHELL is set, new input will be interpreted by the shell immediately.
#
uint8[<=128] input

---

#
# Exit status of the last executed process, or error code of the shell itself.
# Default value is zero.
#
int32 last_exit_status

#
# These flags indicate the status of the shell.
#
uint8 FLAG_RUNNING              = 1     # The shell is currently running a process; stdin/out/e
uint8 FLAG_SHELL_ERROR          = 2     # Exit status contains error code, output contains text
uint8 FLAG_HAS_PENDING_STDOUT   = 64    # There is more stdout to read
uint8 FLAG_HAS_PENDING_STDERR   = 128   # There is more stderr to read
uint8 flags

#
# In case of a shell error, this string may contain ASCII string explaining the nature of the e
# Otherwise, if stdout read is requested, this string will contain stdout data. If stderr read
# this string will contain stderr data. If both stdout and stderr read is requested, this strin
# with stdout and end with stderr, with no separator in between.
#
uint8[<=256] output
```

CANIfaceStats

Full name: `uavcan.protocol.CANIfaceStats`

```
#
# Single CAN iface statistics.
#

uint48 frames_tx        # Number of transmitted CAN frames.
uint48 frames_rx        # Number of received CAN frames.
uint48 errors           # Number of errors in the CAN layer.
```

DataTypeKind

Full name: `uavcan.protocol.DataTypeKind`

```
#
# Data type kind (message or service).
#

uint8 SERVICE = 0
uint8 MESSAGE = 1
uint8 value
```

HardwareVersion

Full name: `uavcan.protocol.HardwareVersion`

```
#
# Nested type.
# Generic hardware version information.
# These values should remain unchanged for the device's lifetime.
#

#
# Hardware version code.
#
uint8 major
uint8 minor

#
# Unique ID is a 128 bit long sequence that is globally unique for each node.
# All zeros is not a valid UID.
# If filled with zeros, assume that the value is undefined.
#
uint8[16] unique_id

#
# Certificate of authenticity (COA) of the hardware, 255 bytes max.
#
uint8[<=255] certificate_of_authenticity
```

SoftwareVersion

Full name: `uavcan.protocol.SoftwareVersion`

```
#
# Nested type.
# Generic software version information.
#

#
# Primary version numbers.
# If both fields are set to zero, the version is considered unknown.
#
uint8 major
uint8 minor

#
# This mask indicates which optional fields (see below) are set.
#
uint8 OPTIONAL_FIELD_FLAG_VCS_COMMIT = 1
uint8 OPTIONAL_FIELD_FLAG_IMAGE_CRC  = 2
uint8 optional_field_flags

#
# VCS commit hash or revision number, e.g. git short commit hash. Optional.
#
uint32 vcs_commit

#
# The value of an arbitrary hash function applied to the firmware image.
# This field is used to detect whether the firmware running on the node is EXACTLY THE SAME
# as a certain specific revision. This field provides the absolute identity guarantee, unlike
# the version fields above, which can be the same for different builds of the firmware.
#
# The exact hash function and the methods of its application are implementation defined.
# However, implementations are recommended to adhere to the following guidelines,
# fully or partially:
#
#   - The hash function should be CRC-64-WE, the same that is used for computing DSDL signature
```

```
#   - If the computed image CRC is stored within the firmware image itself, the value of
#     the hash function becomes ill-defined, because it becomes recursively dependent on itself
#     In order to circumvent this issue, while computing or checking the CRC, its value stored
#     within the image should be zeroed out.
#
uint64 image_crc
```

uavcan.protocol.debug

KeyValue

Full name: `uavcan.protocol.debug.KeyValue`

Default data type ID: 16370

```
#
# Generic named parameter (key/value pair).
#

#
# Integers are exactly representable in the range (-2^24, 2^24) which is (-16'777'216, 16'777'2
#
float32 value

#
# Tail array optimization is enabled, so if key length does not exceed 3 characters, the whole
# message can fit into one CAN frame. The message always fits into one CAN FD frame.
#
uint8[<=58] key
```

LogMessage

Full name: `uavcan.protocol.debug.LogMessage`

Default data type ID: 16383

```
#
# Generic log message.
# All items are byte aligned.
#

LogLevel level
uint8[<=31] source
uint8[<=90] text
```

LogLevel

Full name: `uavcan.protocol.debug.LogLevel`

```
#
# Log message severity
#

uint3 DEBUG    = 0
uint3 INFO     = 1
uint3 WARNING  = 2
uint3 ERROR    = 3
uint3 value
```

uavcan.protocol.dynamic_node_id

Allocation

Full name: `uavcan.protocol.dynamic_node_id.Allocation`

Default data type ID: 1

```
#
# This message is used for dynamic Node ID allocation.
#
# When a node needs to request a node ID dynamically, it will transmit an anonymous message tra
# In order to reduce probability of CAN ID collisions when multiple nodes are publishing this r
# field of anonymous message transfer includes a Discriminator, which is a special field that h
# random data by the transmitting node. Since Discriminator collisions are likely to happen (pr
# 0.006%), nodes that are requesting dynamic allocations need to be able to handle them correct
# resolution protocol is defined (alike CSMA/CD). The collision resolution protocol is based on
# transmission intervals:
#
# - Request period - Trequest.
# - Follow up delay - Tfollowup.
#
# Recommended randomization ranges for these intervals are documented in the constants of this
# Random intervals must be chosen anew per transmission, whereas the Discriminator value is all
# per node.
#
# In the below description the following terms are used:
# - Allocator - the node that serves allocation requests.
# - Allocatee - the node that requests an allocation from the Allocator.
#
# The response timeout is not explicitly defined for this protocol, as the Allocatee will reque
# Trequest units of time later again, unless the allocation has been granted. Despite this, the
# consider the value of FOLLOWUP_TIMEOUT_MS as an allocation timeout, if necessary.
#
# On the allocatee's side the protocol is defined through the following set of rules:
#
# Rule A. On initialization:
# 1. The allocatee subscribes to this message.
# 2. The allocatee starts the Request Timer with a random interval of Trequest.
#
# Rule B. On expiration of Request Timer:
# 1. Request Timer restarts with a random interval of Trequest.
# 2. The allocatee broadcasts a first-stage Allocation request message, where the fields are as
#    node_id                - preferred node ID, or zero if the allocatee doesn't have any pre
#    first_part_of_unique_id - true
#    unique_id              - first MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST bytes of unique ID
#
# Rule C. On any Allocation message, even if other rules also match:
# 1. Request Timer restarts with a random interval of Trequest.
#
# Rule D. On an Allocation message WHERE (source node ID is non-anonymous) AND (allocatee's uni
# bytes available in the field unique_id) AND (unique_id is less than 16 bytes long):
# 1. The allocatee waits for Tfollowup units of time, while listening for other Allocation mess
#    message is received during this time, the execution of this rule will be terminated. Also
# 2. The allocatee broadcasts a second-stage Allocation request message, where the fields are a
#    node_id                - same value as in the first-stage
#    first_part_of_unique_id - false
#    unique_id              - at most MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST bytes of local unique
#                             equal to number of bytes in the received unique ID
#
# Rule E. On an Allocation message WHERE (source node ID is non-anonymous) AND (unique_id fully
# unique ID) AND (node_id in the received message is not zero):
# 1. Request Timer stops.
# 2. The allocatee initializes its node_id with the received value.
# 3. The allocatee terminates subscription to Allocation messages.
# 4. Exit.
#
```

```
# These definitions have an advisory status; it is OK to pick higher values for both bounds, as
# protocol compatibility. In fact, it is advised to pick higher values if the target applicatio
# about the time it will spend on completing the dynamic node ID allocation procedure, as it wi
# interference with other nodes, possibly of higher importance.
#
# The lower bound shall not be lower than FOLLOWUP_TIMEOUT_MS, otherwise the request may confli
#
uint16 MAX_REQUEST_PERIOD_MS = 1000     # It is OK to exceed this value
uint16 MIN_REQUEST_PERIOD_MS = 600      # It is OK to exceed this value

#
# Recommended randomization range for followup delay.
# The upper bound shall not exceed FOLLOWUP_TIMEOUT_MS, because the allocator will reset the st
#
uint16 MAX_FOLLOWUP_DELAY_MS = 400
uint16 MIN_FOLLOWUP_DELAY_MS = 0        # Defined only for regularity; will always be zero.

#
# Allocator will reset its state if there was no follow-up request in this amount of time.
#
uint16 FOLLOWUP_TIMEOUT_MS = 500

#
# Any request message can accommodate no more than this number of bytes of unique ID.
# This limitation is needed to ensure that all request transfers are single-frame.
# This limitation does not apply to CAN FD transport.
#
uint8 MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST = 6

#
# When requesting an allocation, set the field 'node_id' to this value if there's no preference
#
uint7 ANY_NODE_ID = 0

#
# If transfer is anonymous, this is the preferred ID.
# If transfer is non-anonymous, this is allocated ID.
#
# If the allocatee does not have any preference, this value must be set to zero. In this case,
# must choose the highest unused node ID value for this allocation (except 126 and 127, that ar
# network maintenance tools). E.g., if the allocation table is empty and the node has requested
# without any preference, the allocator will grant the node ID 125.
#
# If the preferred node ID is not zero, the allocator will traverse the allocation table starti
# preferred node ID upward, until a free node ID is found. If a free node ID could not be found
# allocator will restart the search from the preferred node ID downward, until a free node ID i
#
# In pseudocode:
#   int findFreeNodeID(const int preferred)
#   {
#       // Search up
#       int candidate = (preferred > 0) ? preferred : 125;
#       while (candidate <= 125)
#       {
#           if (!isOccupied(candidate))
#               return candidate;
#           candidate++;
#       }
#       // Search down
#       candidate = (preferred > 0) ? preferred : 125;
#       while (candidate > 0)
#       {
#           if (!isOccupied(candidate))
#               return candidate;
#           candidate--;
#       }
#       // Not found
#       return -1;
#   }
#
uint7 node_id
```

```
#
bool first_part_of_unique_id

#
# If transfer is anonymous, this array must not contain more than MAX_LENGTH_OF_UNIQUE_ID_IN_RE
# Note that array is tail-optimized, i.e. it will not be prepended with length field.
#
uint8[<=16] unique_id
```

uavcan.protocol.dynamic_node_id.server

AppendEntries

Full name: `uavcan.protocol.dynamic_node_id.server.AppendEntries`

Default data type ID: 30

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This type is a part of the Raft consensus algorithm.
# Please refer to the specification for details.
#

#
# Given min election timeout and cluster size, the maximum recommended request interval can be
#
#   max recommended request interval = (min election timeout) / 2 requests / (cluster size - 1)
#
# The equation assumes that the Leader requests one Follower at a time, so that there's at most
# at any moment. Such behavior is optimal as it creates uniform bus load, but it is actually im
# Obviously, request interval can be lower than that if needed, but higher values are not recom
# cause Followers to initiate premature elections in case of intensive frame losses or delays.
#
# Real timeout is randomized in the range (MIN, MAX], according to the Raft paper.
#
uint16 DEFAULT_MIN_ELECTION_TIMEOUT_MS = 2000
uint16 DEFAULT_MAX_ELECTION_TIMEOUT_MS = 4000

#
# Refer to the Raft paper for explanation.
#
uint32 term
uint32 prev_log_term
uint8 prev_log_index
uint8 leader_commit

#
# Worst-case replication time per Follower can be computed as:
#
#   worst replication time = (127 log entries) * (2 trips of next_index) * (request interval pe
#
Entry[<=1] entries

---

#
# Refer to the Raft paper for explanation.
#
uint32 term
bool success
```

RequestVote

Full name: `uavcan.protocol.dynamic_node_id.server.RequestVote`

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This type is a part of the Raft consensus algorithm.
# Please refer to the specification for details.
#

#
# Refer to the Raft paper for explanation.
#
uint32 term
uint32 last_log_term
uint8 last_log_index

---

#
# Refer to the Raft paper for explanation.
#
uint32 term
bool vote_granted
```

`Discovery`

Full name: `uavcan.protocol.dynamic_node_id.server.Discovery`

Default data type ID: 390

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This message is used by allocation servers to find each other's node ID.
# Please refer to the specification for details.
#
# A server should stop publishing this message as soon as it has discovered all other nodes in
#
# An exception applies: when a server receives a Discovery message from another server where th
# of known nodes is incomplete (i.e. len(known_nodes) < configured_cluster_size), the server mu
# publish a discovery message once. This condition allows other servers to quickly re-discover
# after restart.
#

#
# This message should be broadcasted by the server at this interval until all other servers are
#
uint16 BROADCASTING_PERIOD_MS = 1000

#
# Number of servers in the cluster as configured on the sender.
#
uint8 configured_cluster_size

#
# Node ID of servers that are known to the publishing server, including the publishing server i
# Capacity of this array defines maximum size of the server cluster.
#
uint8[<=5] known_nodes
```

`Entry`

Full name: `uavcan.protocol.dynamic_node_id.server.Entry`

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# One dynamic node ID allocation entry.
```

```
uint32 term             # Refer to the Raft paper for explanation.

uint8[16] unique_id     # Unique ID of this allocation.

void1
uint7 node_id           # Node ID of this allocation.
```

uavcan.protocol.enumeration

Begin

Full name: `uavcan.protocol.enumeration.Begin`

Default data type ID: 15

```
#
# This service instructs the node to begin the process of automated enumeration.
#

#
# The node will automatically leave enumeration mode upon expiration of this timeout.
#
uint16 TIMEOUT_CANCEL   = 0     # Stop enumeration immediately
uint16 TIMEOUT_INFINITE = 65535 # Do not stop until explicitly requested
uint16 timeout_sec              # [Seconds]

#
# Name of the parameter to enumerate, e.g. ESC index.
# If the name is left empty, the node will infer the parameter name automatically (autodetect).
# It is highly recommended to always use autodetection in order to avoid dependency on hard-cod
# and also allow the enumeratee to possibly enumerate multiple different parameters at once.
# The rule of thumb is to always leave this parameter empty unless you really know what you're
#
uint8[<=92] parameter_name

---

uint8 ERROR_OK                  = 0     # Success
uint8 ERROR_INVALID_MODE        = 1     # The node cannot perform enumeration in its current op
uint8 ERROR_INVALID_PARAMETER   = 2     # The node cannot enumerate on the requested parameter,
uint8 ERROR_UNSUPPORTED         = 3     # The node cannot perform enumeration in its current co
uint8 ERROR_UNKNOWN             = 255   # Generic error
uint8 error
```

Indication

Full name: `uavcan.protocol.enumeration.Indication`

Default data type ID: 380

```
#
# This message will be broadcasted when the node receives user input in the process of enumerat
#

void6

#
# This field is unused; keep it empty
#
uavcan.protocol.param.NumericValue value

#
# Name of the enumerated parameter.
```

```
#
uint8[<=92] parameter_name
```

uavcan.protocol.file

BeginFirmwareUpdate

Full name: `uavcan.protocol.file.BeginFirmwareUpdate`

Default data type ID: 40

```
#
# This service initiates firmware update on a remote node.
#
# The node that is being updated (slave) will retrieve the firmware image file 'image_file_remo
# 'source_node_id' using the file read service, then it will update the firmware and reboot.
#
# The slave can explicitly reject this request if it is not possible to update the firmware at
# (e.g. if the node is busy).
#
# If the slave node accepts this request, the initiator will get a response immediately, before
# actually begins.
#
# While the firmware is being updated, the slave should set its mode (uavcan.protocol.NodeStatu
# MODE_SOFTWARE_UPDATE.
#

uint8 source_node_id         # If this field is zero, the caller's Node ID will be used instead
Path image_file_remote_path

---

#
# Other error codes may be added in the future.
#
uint8 ERROR_OK               = 0
uint8 ERROR_INVALID_MODE     = 1    # Cannot perform the update in the current operating mode o
uint8 ERROR_IN_PROGRESS      = 2    # Firmware update is already in progress, and the slave doe
uint8 ERROR_UNKNOWN          = 255
uint8 error

uint8[<128] optional_error_message   # Detailed description of the error.
```

GetInfo

Full name: `uavcan.protocol.file.GetInfo`

Default data type ID: 45

```
#
# Request info about a remote file system entry (file, directory, etc).
#

Path path

---

#
# File size in bytes.
# Should be set to zero for directories.
#
uint40 size

Error error
```

GetDirectoryEntryInfo

Full name: uavcan.protocol.file.GetDirectoryEntryInfo

Default data type ID: 46

```
#
# This service can be used to retrieve a remote directory listing, one entry per request.
#
# The client should query each entry independently, iterating 'entry_index' from 0 until the la
# in which case the server will report that there is no such entry (via the fields 'entry_type'
#
# The entry_index shall be applied to the ordered list of directory entries (e.g. alphabeticall
# sorting criteria does not matter as long as it provides the same ordering for subsequent serv
#

uint32 entry_index

Path directory_path

---

Error error

EntryType entry_type

Path entry_full_path  # Ignored/Empty if such entry does not exist.
```

Delete

Full name: uavcan.protocol.file.Delete

Default data type ID: 47

```
#
# Delete remote file system entry.
# If the remote entry is a directory, all nested entries will be removed too.
#

Path path

---

Error error
```

Read

Full name: uavcan.protocol.file.Read

Default data type ID: 48

```
#
# Read file from a remote node.
#
# There are two possible outcomes of a successful service call:
#  1. Data array size equals its capacity. This means that the end of the file is not reached y
#  2. Data array size is less than its capacity, possibly zero. This means that the end of file
#
# Thus, if the client needs to fetch the entire file, it should repeatedly call this service wh
# offset, until incomplete data is returned.
#
# If the object pointed by 'path' cannot be read (e.g. it is a directory or it does not exist),
```

```
uint40 offset

Path path

---

Error error

uint8[<=256] data
```

`Write`

Full name: `uavcan.protocol.file.Write`

Default data type ID: 49

```
#
# Write into a remote file.
# The server shall place the contents of the field 'data' into the file pointed by 'path' at th
# the field 'offset'.
#
# When writing a file, the client should repeatedly call this service with data while advancing
# is written completely. When write is complete, the client shall call the service one last tim
# set to the size of the file and with the data field empty, which will signal the server that
# complete.
#
# When the write operation is complete, the server shall truncate the resulting file past the s
#
# Server implementation advice:
# It is recommended to implement proper handling of concurrent writes to the same file from dif
# example by means of creating a staging area for uncompleted writes (like FTP servers do).
#

uint40 offset

Path path

uint8[<=192] data

---

Error error
```

`EntryType`

Full name: `uavcan.protocol.file.EntryType`

```
#
# Nested type.
# Represents the type of the file system entry (e.g. file or directory).
# If such entry does not exist, 'flags' must be set to zero.
#

uint8 FLAG_FILE      = 1        # Excludes FLAG_DIRECTORY
uint8 FLAG_DIRECTORY = 2        # Excludes FLAG_FILE
uint8 FLAG_SYMLINK   = 4        # Link target is either FLAG_FILE or FLAG_DIRECTORY
uint8 FLAG_READABLE  = 8
uint8 FLAG_WRITEABLE = 16

uint8 flags
```

`Error`

```
#
# Nested type.
# File operation result code.
#

int16 OK                = 0
int16 UNKNOWN_ERROR     = 32767

int16 NOT_FOUND         = 2
int16 IO_ERROR          = 5
int16 ACCESS_DENIED     = 13
int16 IS_DIRECTORY      = 21 # I.e. attempt to read/write on a path that points to a directory
int16 INVALID_VALUE     = 22 # E.g. file name is not valid for the target file system
int16 FILE_TOO_LARGE    = 27
int16 OUT_OF_SPACE      = 28
int16 NOT_IMPLEMENTED   = 38

int16 value
```

Path

Full name: `uavcan.protocol.file.Path`

```
#
# Nested type.
#
# File system path in UTF8.
#
# The only valid separator is forward slash.
#

uint8 SEPARATOR = '/'

uint8[<=200] path
```

uavcan.protocol.param

ExecuteOpcode

Full name: `uavcan.protocol.param.ExecuteOpcode`

Default data type ID: 10

```
#
# Service to control the node configuration.
#

#
# SAVE operation instructs the remote node to save the current configuration parameters into a
# storage. The node may require a restart in order for some changes to take effect.
#
# ERASE operation instructs the remote node to clear its configuration storage and reinitialize
# with their default values. The node may require a restart in order for some changes to take e
#
# Other opcodes may be added in the future (for example, an opcode for switching between multip
#
uint8 OPCODE_SAVE  = 0  # Save all parameters to non-volatile storage.
uint8 OPCODE_ERASE = 1  # Clear the non-volatile storage; some changes may take effect only aft
uint8 opcode

#
# Reserved, keep zero.
#
int48 argument
```

```
# If 'ok' (the field below) is true, this value is not used and must be kept zero.
# If 'ok' is false, this value may contain error code. Error code constants may be defined in t
#
int48 argument

#
# True if the operation has been performed successfully, false otherwise.
#
bool ok
```

GetSet

Full name: `uavcan.protocol.param.GetSet`

Default data type ID: 11

```
#
# Get or set a parameter by name or by index.
# Note that access by index should only be used to retrieve the list of parameters; it is highl
# discouraged to use it for anything else, because persistent ordering is not guaranteed.
#

#
# Index of the parameter starting from 0; ignored if name is nonempty.
# Use index only to retrieve the list of parameters.
# Parameter ordering must be well defined (e.g. alphabetical, or any other stable ordering),
# in order for the index access to work.
#
uint13 index

#
# If set - parameter will be assigned this value, then the new value will be returned.
# If not set - current parameter value will be returned.
# Refer to the definition of Value for details.
#
Value value

#
# Name of the parameter; always preferred over index if nonempty.
#
uint8[<=92] name

---

void5

#
# Actual parameter value.
#
# For set requests, it should contain the actual parameter value after the set request was
# executed. The objective is to let the client know if the value could not be updated, e.g.
# due to its range violation, etc.
#
# Empty value (and/or empty name) indicates that there is no such parameter.
#
Value value

void5
Value default_value    # Optional

void6
NumericValue max_value # Optional, not applicable for bool/string

void6
NumericValue min_value # Optional, not applicable for bool/string

#
# Empty name (and/or empty value) in response indicates that there is no such parameter.
#
```

Empty

Full name: `uavcan.protocol.param.Empty`

```
#
# Ex nihilo nihil fit.
#
```

NumericValue

Full name: `uavcan.protocol.param.NumericValue`

```
#
# Numeric-only value.
#
# This is a union, which means that this structure can contain either one of the fields below.
# The structure is prefixed with tag - a selector value that indicates which particular field i
#

@union                          # Tag is 2 bits long.

Empty empty                     # Empty field, used to represent an undefined value.

int64   integer_value
float32 real_value
```

Value

Full name: `uavcan.protocol.param.Value`

```
#
# Single parameter value.
#
# This is a union, which means that this structure can contain either one of the fields below.
# The structure is prefixed with tag - a selector value that indicates which particular field i
#

@union                          # Tag is 3 bit long, so outer structure has 5-bit prefix to ens

Empty empty                     # Empty field, used to represent an undefined value.

int64        integer_value
float32      real_value         # 32-bit type is used to simplify implementation on low-end sys
uint8        boolean_value      # 8-bit value is used for alignment reasons
uint8[<=128] string_value       # Length prefix is exactly one byte long, which ensures proper
```

uavcan.tunnel

Broadcast

Full name: `uavcan.tunnel.Broadcast`

Default data type ID: 2010

```
#
# This message struct carries arbitrary data in the format of the specified high-level protocol
# The data will be delivered to all nodes that are interested in tunneled protocols.
# Finer addressing schemes may be implemented using the means provided by the encapsulated prot
# The channelID allows for additional routing between the source and target nodes.
```

```
uint8[<=60] buffer    # TAO rules apply
```

Call

Full name: `uavcan.tunnel.Call`

Default data type ID: 63

```
#
# This service carries arbitrary data in the format of the specified high-level protocol.
# The data will be delivered to the specified node only (not broadcast), and the addressed node
# will be required to respond (although the response may be empty, if the chosen protocol allow
# The specified protocol applies both to the request and to the response. The channelID allows
# additional routing between the source and target nodes.
#

Protocol protocol
uint8 channel_id

uint8[<=60] buffer    # TAO rules apply

---

uint8[<=60] buffer    # TAO rules apply
```

Protocol

Full name: `uavcan.tunnel.Protocol`

```
#
# This enumeration specifies the encapsulated protocol.
# New protocols are likely to be added in the future.
#

uint8 MAVLINK                   = 0     # MAVLink

uint8 protocol
```

dronecan.sensors.hygrometer

Hygrometer

Full name: `dronecan.sensors.hygrometer.Hygrometer`

Default data type ID: 1032

```
#
# support for hygrometer sensors
#

# temperature in degrees C, use NaN if not available
float16 temperature

# humidity as percentage
float16 humidity

# id of humidity sensor within this CAN node. Use 0 for first sensor
uint8 id
```

ardupilot.equipment.power

BatteryInfoAux

Full name: `ardupilot.equipment.power.BatteryInfoAux`

Default data type ID: 20004

```
#
# Auxilary Single battery info.
#
# This message is to be used in conjunction with uavcan/equipment/power/1092.BatteryInfo.uavcan

# Please ensure that this message is sent before the BatteryInfo which doesn't
# include the timestamp. The following fields should contain the timestamp of the last current
uavcan.Timestamp timestamp

# Battery individual cell voltages
# length of following field also used as cell count
float16[<=255] voltage_cell          # [Volt]

# Number of Charge/Discharge Cycles
# A charge cycle is defined as a complete discharge and charge.
# IE when the system uses all of the battery's available energy capacity.
# Please note that this value doesn't mean that the cycle count should be incremented at every
# For example, if half of a battery's charge is used in one run,
# and then recharged fully that is half of a cycle. If the same thing occurs again
# then the charge cycle count would be incremented once, not twice.
uint16 cycle_count

# Number of times the battery was discharged over the rated capacity
uint16 over_discharge_count

# Max instantaneous current draw since last message
float16 max_current                 # [Ampere]

# Nominal voltage of the battery pack
# the nominal Voltage can be used for conversion between Wh and Ah
# if the value of this field is 0, the conversion should be omitted.
float16 nominal_voltage             # [Volt]

bool is_powering_off                # Power off event imminent indication, false if unknown

uint8 battery_id                    # Identifies the battery within this vehicle, e.g. 0 - prim
```

ardupilot.equipment.trafficmonitor

TrafficReport

Full name: `ardupilot.equipment.trafficmonitor.TrafficReport`

Default data type ID: 20790

```
# Network-synchronized timestamp, 0 if not available. Note: not necessarily a UTC time.
uavcan.Timestamp timestamp

# icao address
uint32 icao_address

# Time since last communication in seconds
uint16 tslc

# Traffic position in lat-lon-alt. Check alt_type for altitude datum
int32 latitude_deg_1e7
```

```
# Traffic heading in radians.
# Course over ground if heading is unavailable. 0 if neither are available.
float16 heading

# Traffic velocity in m/s
float16[3] velocity

# Traffic squawk code
uint16 squawk

# Traffic callsign
uint8[9] callsign

# Traffic source
uint3 SOURCE_ADSB = 0
uint3 SOURCE_ADSB_UAT = 1
uint3 SOURCE_FLARM = 2
uint3 source

# Traffic type
uint5 TRAFFIC_TYPE_UNKNOWN = 0
uint5 TRAFFIC_TYPE_LIGHT = 1
uint5 TRAFFIC_TYPE_SMALL = 2
uint5 TRAFFIC_TYPE_LARGE = 3
uint5 TRAFFIC_TYPE_HIGH_VORTEX_LARGE = 4
uint5 TRAFFIC_TYPE_HEAVY = 5
uint5 TRAFFIC_TYPE_HIGHLY_MANUV = 6
uint5 TRAFFIC_TYPE_ROTOCRAFT = 7
uint5 TRAFFIC_TYPE_GLIDER = 9
uint5 TRAFFIC_TYPE_LIGHTER_THAN_AIR = 10
uint5 TRAFFIC_TYPE_PARACHUTE = 11
uint5 TRAFFIC_TYPE_ULTRA_LIGHT = 12
uint5 TRAFFIC_TYPE_UAV = 14
uint5 TRAFFIC_TYPE_SPACE = 15
uint5 TRAFFIC_TYPE_EMERGENCY_SURFACE = 17
uint5 TRAFFIC_TYPE_SERVICE_SURFACE = 18
uint5 TRAFFIC_TYPE_POINT_OBSTACLE = 19
uint5 traffic_type

# Altitude type
uint7 ALT_TYPE_ALT_UNKNOWN = 0
uint7 ALT_TYPE_PRESSURE_AMSL = 1
uint7 ALT_TYPE_WGS84 = 2
uint7 alt_type

# Validity flags
bool lat_lon_valid
bool heading_valid
bool velocity_valid
bool callsign_valid
bool ident_valid
bool simulated_report
bool vertical_velocity_valid
bool baro_valid
```

`ardupilot.gnss`

`Heading`

Full name: `ardupilot.gnss.Heading`

Default data type ID: 20002

```
bool heading_valid
bool heading_accuracy_valid

float16 heading_rad
```

Status

Full name: `ardupilot.gnss.Status`

Default data type ID: 20003

```
# Node specific GNSS error codes, primarily available for logging and diagnostics
uint32 error_codes

# GNSS system is self assesd as healthy
bool healthy

# Status is actually a bitmask, due to encoding issues pretend it's just a field, and leave it
uint23 STATUS_LOGGING = 1  # GNSS system is doing any onboard logging
uint23 STATUS_ARMABLE = 2  # GNSS system is in a reasonable state to allow the system to arm
uint23 status
```

MovingBaselineData

Full name: `ardupilot.gnss.MovingBaselineData`

Default data type ID: 20005

```
# length of data is set per the number of bytes for pkt in
# libraries/AP_GPS/RTCM3_Parser.h
uint8[<=300] data
```

RelPosHeading

Full name: `ardupilot.gnss.RelPosHeading`

Default data type ID: 20006

```
# timestamp on the gps message
uavcan.Timestamp timestamp

bool    reported_heading_acc_available
float32 reported_heading_deg
float32 reported_heading_acc_deg
float16 relative_distance_m
float16 relative_down_pos_m
```

ardupilot.indication

SafetyState

Full name: `ardupilot.indication.SafetyState`

Default data type ID: 20000

```
#
# support for Safety LED on UAVCAN

uint8 STATUS_SAFETY_ON           = 0
uint8 STATUS_SAFETY_OFF          = 255

uint8 status
```

`Button`

Full name: `ardupilot.indication.Button`

Default data type ID: 20001

```
#
# support for buttons on UAVCAN
# while a button is being pressed this message should be sent at 10Hz

uint8 BUTTON_SAFETY      = 1

uint8 button

# number of 0.1s units the button has been pressed for. If over 255
# then send 255
uint8 press_time
```

`NotifyState`

Full name: `ardupilot.indication.NotifyState`

Default data type ID: 20007

```
uint8 VEHICLE_STATE_INITIALISING    = 0
uint8 VEHICLE_STATE_ARMED           = 1
uint8 VEHICLE_STATE_FLYING          = 2
uint8 VEHICLE_STATE_PREARM          = 3
uint8 VEHICLE_STATE_PREARM_GPS      = 4
uint8 VEHICLE_STATE_SAVE_TRIM       = 5
uint8 VEHICLE_STATE_ESC_CALIBRATION = 6
uint8 VEHICLE_STATE_FAILSAFE_RADIO  = 7
uint8 VEHICLE_STATE_FAILSAFE_BATT   = 8
uint8 VEHICLE_STATE_FAILSAFE_GCS    = 9
uint8 VEHICLE_STATE_CHUTE_RELEASED  = 10
uint8 VEHICLE_STATE_EKF_BAD         = 11
uint8 VEHICLE_STATE_FW_UPDATE       = 12
uint8 VEHICLE_STATE_MAGCAL_RUN      = 13
uint8 VEHICLE_STATE_LEAK_DET        = 14
uint8 VEHICLE_STATE_GPS_FUSION      = 15
uint8 VEHICLE_STATE_GPS_GLITCH      = 16
uint8 VEHICLE_STATE_POS_ABS_AVAIL   = 17
uint8 VEHICLE_STATE_LOST            = 18
uint8 VEHICLE_STATE_THROW_READY     = 19
uint8 VEHICLE_STATE_POWERING_OFF    = 20
uint8 VEHICLE_STATE_VIDEO_RECORDING = 21


uint8 VEHICLE_YAW_EARTH_CENTIDEGREES = 0

# set the type of aux data to be sent
uint8 aux_data_type

# bytes containing packed auxiliary data
# placing it first so we don't do TAO
uint8[<=255] aux_data

# Current Vehicle State Mask
uint64 vehicle_state
```

`com.hex.equipment.flow`

`Measurement`

Default data type ID: 20200

```
float32 integration_interval    # Integration Interval in seconds
float32[2] rate_gyro_integral   # Integrated Gyro Data in radians
float32[2] flow_integral        # Integrated LOS Data in radians
uint8 quality                   # Flow Data Quality Lowest(0)-Highest(255) Unitless
```